



Week 3 Workshop Solutions

Pre-Processing: Stemming and Python Classes

IFN647

School of Computer Science
Queensland University of Technology

Workshop Task 1

Task 1: Update Task 3 of last week's workshop to print the terms of the document and their frequency in ascending order. Note that dictionaries cannot be sorted, but you can get a representation of a dictionary that is sorted. Try the following commands:

```
>>> x = {1: 2, 3: 4, 4: 3, 2: 1, 0: 0}
>>> {k: v for k, v in sorted(x.items(), key=lambda item:
item[1])}
>>> {k: v for k, v in sorted(x.items(), reverse=False)}
>>> {k: v for k, v in sorted(x.items(), reverse=True)}
```

Output:

```
addition : 1
aires : 1
amid : 1
argentina : 2
argentine : 1
awaiting : 1
axel : 1
bank : 1
between : 1
bocon : 1
bonds : 1
bounce : 2
buenos : 1
bugge : 1
change : 1
congress : 1
deficit : 1
delegation : 1
denominated : 1
dollar : 1
due : 2
during : 1
early : 1
economic : 1
economy : 1
events : 1
expect : 1
expected : 2
fernandez : 1
fiscal : 1
foreign : 1
```

Time: 10 mins

Task 1

solution

```
import glob, os
import string
#from stemming.porter2 import stem ## for week 3

def parse_doc(inputpath, stop_ws):
    coll = {}
    os.chdir(inputpath)
    #myfile=open('741299newsML.xml')
    myfile=open('6146.xml')
    #myfile=open('C:\\python27\\py_CAB431_201\\data\\741299newsML.xml', 'r')
    curr_doc = {}
    start_end = False
    #docid='741299'
    file_=myfile.readlines()
    word_count = 0 #wk2
    for line in file_:
        line = line.strip()
        #print(line)
        if(start_end == False):
            if line.startswith("<newsitem "):
                for part in line.split():
                    if part.startswith("itemid="):
                        docid = part.split("=")[1].split("\\")[1]
                        break
            if line.startswith("<text>"):
                start_end = True
        elif line.startswith("</text>"):
            break
        else:
            line = line.replace("<p>", "").replace("</p>", "")
            line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
            #print(line)
            #print('-----')
            line = line.replace("\\s+", " ")
            for term in line.split():
                word_count += 1 #wk2
                #term = stem(term.lower()) ## for wk 3
                term = term.lower() #wk2
                if len(term) > 2 and term not in stop_words: #wk2
                    try:
                        curr_doc[term] += 1
                    except KeyError:
                        curr_doc[term] = 1
    myfile.close()
    return(word_count, {docid:curr_doc})
```

return(word_count, {docid:curr_doc}) ←Returning tuple

return a tuple, the first element is the number of words in <text> and
the second one is a dictionary that includes only one pair of doc_id and a dictionary of term_frequency pairs

Task 1 solution cont.

```
if __name__ == '__main__':

    import sys
    if len(sys.argv) != 2:
        sys.stderr.write("USAGE: %s <coll-file>\n" % sys.argv[0])
        sys.exit()

    stopwords_f = open('common-english-words.txt', 'r') # wk2
    stop_words = stopwords_f.read().split(',')
    stopwords_f.close()

    x = parse_doc(sys.argv[1], stop_words)
    #for doc in x[1].items():
    #    print('Document itemid: ' + doc[0] + ' contains: ' + str(x[0]) + ' words and ' + str(len(doc[1])) + ' terms')

    print('-----Terms and Their Frequencies-----')
    for doc in x[1].items():
        doc1 = {k: v for k, v in sorted(doc[1].items(), reverse=False)}
        for term, freq in doc1.items():
            print(term + ' : ' + str(freq))
```

Tasks 2 to 4

Task 2. Use porter2 stemming algorithm to update your last week function *parse_doc*(input, stops) to make sure all terms (words) are stemmed.

Task 3. Define a *Doc_Node* Class and a linked list class *List_Docs*.

Task 4. Design a main function to read a set of xml files and represent each file as a node, then create a linked list to link all nodes together.

Hint: You may design your program by considering all three tasks together. For example,

- *parse_doc*() can be implemented to return the information required (e.g., a *Doc_Node* object) for Tasks 3 and 4.
 - For each document, you can extract the document ID (e.g., 6146) and compute the number of terms using a simple operation, such as *len(p.data[1])* (refer to the *lprint*() function on the next page).

Solutions for Tasks 2 to 4

```
import glob, os
import string
from stemming.porter2 import stem ## for week 3 (wk3)
```

←Task 2

```
#Node class of document
class Doc_Node:
    def __init__(self, data, next=None):
        self.data=data
        self.next=next
```

←Task 3

```
#Linked List calss
class List_Docs:
    def __init__(self, hnode):
        self.head=hnode

    def insert(self, nnode):
        if self.head != None:
            p = self.head
            while p.next != None:
                p=p.next
            p.next=nnode

    def lprint(self):
        if self.head != None:
            p = self.head
            while p!= None:
                print('(ID-'+p.data[0] + ':',end=" ")
                print(str(len(p.data[1]))+' terms)',end=" ")
                if p.next != None:
                    print ('-->', end=" ")
                    p=p.next
            else:
                print('The list is empty!')
```

←len(p.data[1]) is the number of terms of the current document referenced by p.

Solutions for Tasks 2 to 4 cont.

```
def parse_doc(fn, stop_ws):
    coll = {}
    #os.chdir(inputpath)
    #myfile=open('741299newsML.xml')
    myfile=open(fn)
    #myfile=open('C:\\python27\\py_CAB431_201\\data\\741299newsML.xml', 'r')
    curr_doc = {}
    start_end = False
    #docid='741299'
    file_=myfile.readlines()
    #word_count = 0 #wk2
    for line in file_:
        line = line.strip()
        #print(line)
        if(start_end == False):
            if line.startswith("<newsitem "):
                for part in line.split():
                    if part.startswith("itemid="):
                        docid = part.split("=")[1].split("\\")[1]
                        break
            if line.startswith("<text>"):
                start_end = True
        elif line.startswith("</text>"):
            break
        else:
            line = line.replace("<p>", "").replace("</p>", "")
            line = line.translate(str.maketrans('','', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
            line = line.replace("\\s+", " ")
            for term in line.split():
                #word_count += 1 #wk2
                term = stem(term.lower()) ## for wk3
                #term = term.lower() #wk2
                if len(term) > 2 and term not in stop_words: #wk2
                    try:
                        curr_doc[term] += 1
                    except KeyError:
                        curr_doc[term] = 1
    myfile.close()
    dn=Doc_Node((docid,curr_doc), None)
    return(dn)
```

←Return a Doc_Node object initialized with a tuple as its data. The tuple consists of two elements: the first element is the document ID, and the second element is a dictionary containing term–frequency pairs.

Solution for Task 4

```
if __name__ == '__main__':

    import sys
    if len(sys.argv) != 2:
        sys.stderr.write("USAGE: %s <coll-file>\n" % sys.argv[0])
        sys.exit()

    stopwords_f = open('common-english-words.txt', 'r') # wk2
    stop_words = stopwords_f.read().split(',')
    stopwords_f.close()
    os.chdir(sys.argv[1]) # Customized parameter is the data folder name
    fn1='6146.xml'
    fn2='741299newsML.xml'
    fn3='5650.xml'
    # test Task 2
    xn = parse_doc(fn1,stop_words)
    print('-----Stems and Their Frequencies in doc '+ fn1 + '-----')
    doc = (xn.data)[1]
    #doc = {k: v for k, v in sorted(doc.items(), reverse=False)}
    for stem1, freq in doc.items(): #cannot use stem as it is a reserved word
        print(stem1 + ' : ' + str(freq))
    #Test Task 3 and Task 4
    xn1 = parse_doc(fn1,stop_words)
    xn2 = parse_doc(fn2,stop_words)
    xn3 = parse_doc(fn3,stop_words)

    ll= List_Docs(xn1)
    ll.insert(xn2)
    ll.insert(xn3)
    print('----- The linked list for docs '+ fn1 + ', ' + fn2 + ' and ' + fn3 + '-----')
    ll.lprint()
```

-----Stems and Their Frequencies in doc 6146.xml-----

argentín	: 1
bond	: 1
slight	: 2
higher	: 1
small	: 1
technic	: 2
bounc	: 2
wednesday	: 1
amid	: 1
low	: 1
volum	: 1
trader	: 3
larg	: 1
foreign	: 1
bank	: 1
open	: 1

----- The linked list for docs 6146.xml, 741299newsML.xml and 5650.xml-----
(ID-6146: 72 terms) --> (ID-741299: 96 terms) --> (ID-5650: 16 terms)